PHẠM THỊ KHÁNH LINH

Coursera - IBM Data Science Professional Certificate

July 26, 2026

Project Background:

- Commercial space company SpaceX advertises Falcon 9 rocket launches at $62 million, significantly cheaper than competitors.
- The cost savings are primarily due to SpaceX's ability to reuse the first stage of the rocket.

Problem Statement:

- Our goal is to predict whether the Falcon 9 first stage will land successfully. Accurately predicting this outcome helps estimate launch costs and provides valuable insights for competing companies.

GitHub Repository URL:
https://github.com/linhptk2920/IBM-Data-Science-Capstone

Methodology:

- Data Collection: Utilized SpaceX REST API and Web Scraping from Wikipedia. Data
- Wrangling: Cleaned data, handled missing values, and applied One-Hot Encoding using Pandas.
- EDA: Conducted Exploratory Data Analysis using SQL (DB2) and Visualization (Matplotlib/Seaborn).
- Interactive Analytics: Built interactive maps with Folium and dashboards with Plotly Dash.
- Predictive Analysis: Trained Logistic Regression, SVM, Decision Tree, and KNN models.

KeyResults:

- The Decision Tree model achieved the highest prediction accuracy on the test data. Payload mass and launch site are significant indicators of landing success.

Data Collection – SpaceX API

Process Overview:

- Requested rocket launch data from the SpaceX REST API using `requests.get()`.
- Converted the JSON response into a Pandas DataFrame using json_normalize. Filtered
- data to include only Falcon 9 launches.
- Replaced missing Payload Mass values with the calculated mean.

GitHub URL for API Notebook:
https://github.com/linhptk2920/IBM-Data-Science-Capstone/blob/main/jupyter-labs-spacex-data-collection-api.ipynb

Data Collection – Web Scraping

Process Overview:

- Requested historical Falcon 9 launch records from Wikipedia.
- Parsed the HTML response using the BeautifulSoup library. Extracted column
- names from HTML table headers (<th>).
- Iterated through table rows (<tr>) and cells (<td>) to construct a structured Pandas DataFrame.

GitHub URL for Web Scraping Notebook:
https://github.com/linhptk2920/IBM-Data-Science-Capstone/blob/main/jupyter-labs-webscraping.ipynb

Data Wrangling Methodology

Cleaning and Processing:

- Calculated the number of successful and failed landings based on the Outcome column.
- Created a new binary target variable Class (1 = Success, 0 = Failure).
- Applied One-Hot Encoding using pd.get_dummies() on categorical variables such as Orbit, LaunchSite, LandingPad, and Serial.
- Cast all numerical features to `float64` to prepare for Machine Learning models.

GitHub URL for Data Wrangling Notebook:
https://github.com/linhptk2920/IBM-Data-Science-Capstone/blob/main/labs-jupyter-spacex-Data-wrangling.ipynb

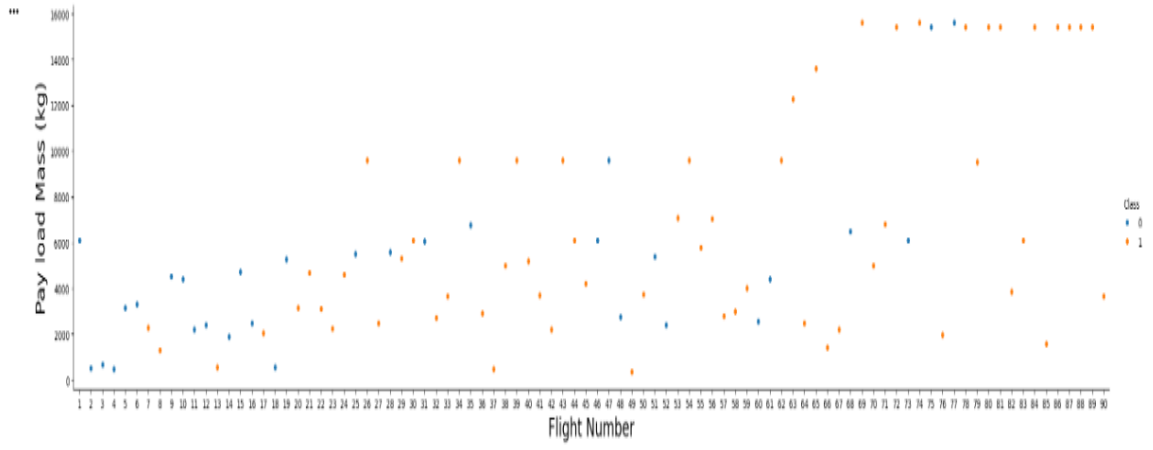
EDA with Data Visualization

Charts Used and Their Purpose:

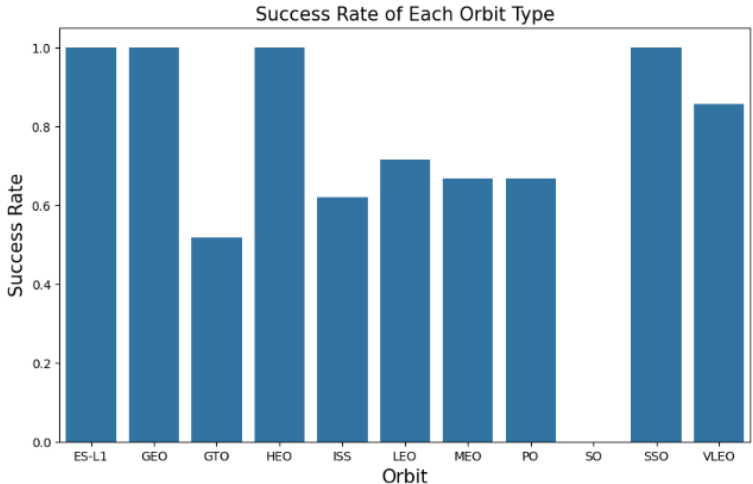
- Scatter Plots: Used to visualize the relationship between *Flight Number vs. Launch Site* and *Payload Mass vs. Launch Site*, colored by landing class.
- Bar Charts: Used to display the success rate across different *Orbit* types.
- Line Charts: Used to track the *success rate over the years* (annual trends).

GitHub URL for EDA Visualization Notebook:
https://github.com/linhptk2920/IBM-Data-Science-Capstone/blob/main/jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA Visualization Results 1



EDA Visualization Results 2



EDA with SQL

- SQL Queries Performed:
  - ✓ Queried unique Launch Sites from the database.
  - ✓ Calculated total Payload Mass carried by boosters.
  - ✓ Counted successful landings at specific sites (e.g., CCAFS).
  - ✓ Retrieved records of specific mission outcomes based on payload mass ranges.
  - ✓ Ranked launch sites by successful landing counts using GROUP BY and ORDER BY.
  - ✓ Performed time analysis to observe successful landing trends over specific months/years
- GitHub URL for SQL Notebook: *https://github.com/linhptk2920/IBM-Data-Science-Capstone/blob/main/jupyter-labs-eda-sql-coursera_sqllite.ipynb*

EDA with SQL Results

Task 1

Display the names of the unique launch sites in the space mission

```
%sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE;
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

EDA with SQL Results

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%sql SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%sql SELECT SUM(PAYLOAD_MASS__KG_) AS Total_Payload_Mass FROM SPACEXTABLE WHERE Customer = 'NASA (CRS)';
```

```
 * sqlite:///my_data1.db
Done.
```

Total_Payload_Mass
45596

EDA with SQL Results

Task 4

Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT AVG(PAYLOAD_MASS__KG_) AS Average_Payload_Mass FROM SPACEXTABLE WHERE Booster_Version = 'F9 v1.1';
```

```
 * sqlite:///my_data1.db
Done.
```

Average_Payload_Mass
2928.4

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
%sql SELECT MIN(Date) AS First_Successful_Landing_Date FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (ground pad)';
```

```
 * sqlite:///my_data1.db
Done.
```

First_Successful_Landing_Date
2015-12-22

EDA with SQL Results

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%sql SELECT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000;
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Task 7

List the total number of successful and failure mission outcomes

```
%sql SELECT Mission_Outcome, COUNT(*) as Total_Number FROM SPACEXTABLE GROUP BY Mission_Outcome;
```

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	Total_Number
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

EDA with SQL Results

Task 8

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%sql SELECT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTABLE);
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

EDA with SQL Results

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%sql SELECT substr(Date, 6, 2) AS Month, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE substr(Date, 1, 4) = '2015' AND Landing_Outcome = 'Failure (drone ship)';
```

```
 * sqlite:///my_data1.db
Done.
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%sql SELECT Landing_Outcome, COUNT(*) as Outcome_Count FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY Outcome_Count DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Interactive Visual Analytics

Folium Map:

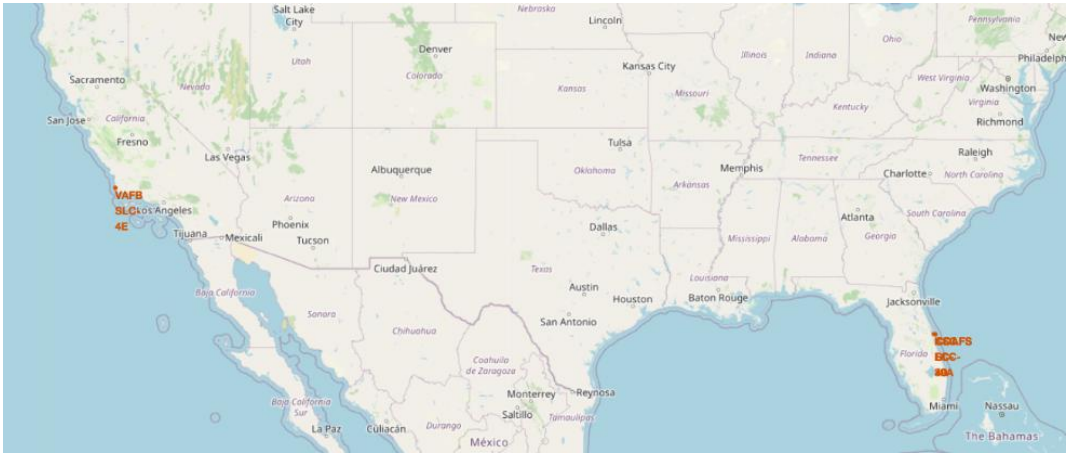
- Plotted Launch Sites on an interactive map.
- Added Marker Clusters to differentiate successful (green) and failed (red) launches.
- Performed proximity analysis by calculating distances to railways, highways, and coastlines.

Plotly Dash App:

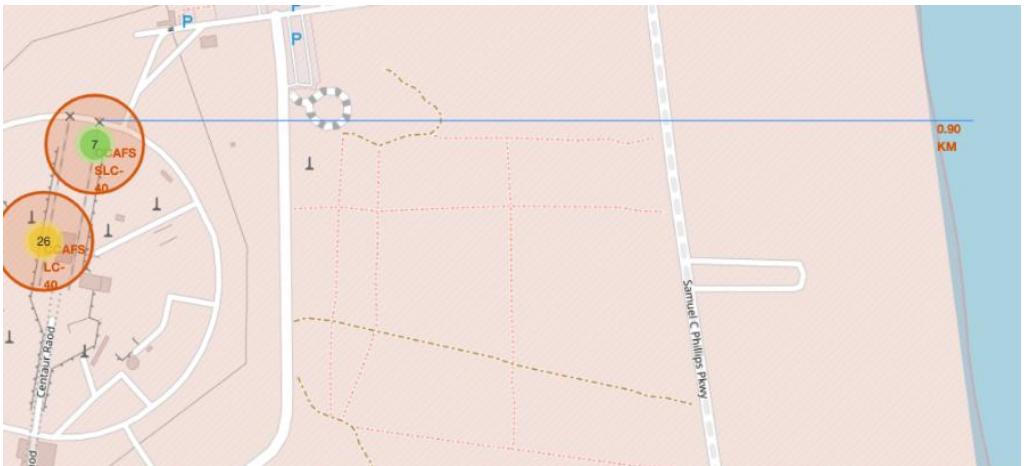
- Built a web dashboard with dropdowns and range sliders.
- Created interactive Pie Charts for success rates by site.
- Created Scatter Plots for Payload Mass vs. Outcome.

GitHub URLs: *https://github.com/linhptk2920/IBM-Data-Science-Capstone/blob/main/lab_jupyter_launch_site_location.ipynb*

Folium Map Results



Folium Map Results



Plotly Dash Results

Predictive Analysis

Model Evaluation & Conclusion:

- Models Tested: Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN).
- Best Model: All models performed similarly well on the test data (approx. 83.3% accuracy), but the Decision Tree (or SVM) is preferred for its specific hyperparameter tuning results.

Predictive Analysis